SEASUG Paper 110-2026

From Annotated CRFs to Define Specifications: A SAS® Macro Automation Framework

Author: Bharath Pappula

ABSTRACT

The preparation of Define specifications from annotated case report forms (aCRFs) is a time-consuming activity that commonly involves several manual steps, including identification of SDTM domains and variables, population of specification metadata, development of value-level metadata, and assignment of source CRF page references. This repetitive process involves repeated transcription of information already available within the aCRF.

This paper presents a SAS® macro-based framework for automating the preparation of an initial Define specification draft from an annotated CRF. The framework uses the aCRF annotation data in FDF format together with the SDTM Implementation Guide and Controlled Terminology as reference inputs. It extracts annotated domains and variables, identifies parent, supplemental qualifier, associated person, and custom domains, validates detected domains, and populates dataset- and variable-level specification metadata. The framework also derives annotated CRF page references and automatically assigns applicable codelists to standard variables. In addition, QNAM- and TESTCD-based annotations are used to generate value-level metadata, with Controlled Terminology providing the corresponding controlled values and labels. The resulting information is organized into standardized Datasets, Variables, and ValueLevel worksheets that support downstream Define-XML development.

A pilot time comparison was used to assess the practical impact of the framework by comparing the time required for first-pass specification preparation using the conventional manual approach versus the automated workflow. The comparison reflects first-pass preparation and does not include final standards review or study-specific programming decisions.

The paper discusses the framework design, implementation, practical application, and validation considerations, and provides practical insights into using SAS to accelerate Define specification development and establish a consistent, traceable starting point for CDISC-compliant regulatory submissions.

Keywords: annotated CRF; aCRF; FDF; SAS macro; Define-XML; SDTM; Controlled Terminology; value-level metadata; automation

INTRODUCTION

Annotated case report forms provide a structured view of how collected clinical data are intended to map to standardized submission domains and variables. The annotations can contain domain names, variable names, value-level conditions, and references to the pages on which the source information occurs. Consequently, the aCRF already contains much of the information needed to begin specification development.

In practice, however, this information is often converted into a Define specification through repeated manual activities. A programmer reviews the annotated document, determines which domains are represented, identifies variables, transcribes source pages, determines whether supplemental or associated-person structures apply, searches the Controlled Terminology for applicable codelists, and prepares value-level metadata. The same source information may be reviewed multiple times while the specification is being built and checked.

The framework described in this paper addresses this gap by treating the annotated CRF as a machine-readable input rather than only as a document for manual review. A PDF annotation export in FDF format is parsed by SAS, combined with SDTM Implementation Guide metadata and Controlled Terminology, and transformed into three specification outputs: Datasets, Variables, and ValueLevel. The framework is intended to generate a traceable initial specification draft rather than replace programming judgment or the final specification review.

FRAMEWORK OVERVIEW

Figure 1 illustrates the major stages implemented by the macro. The workflow begins with the annotated CRF exported to FDF format. The macro then reads and normalizes annotation objects, determines the domains and variables represented in the annotations, enriches the extracted metadata with SDTM Implementation Guide and Controlled Terminology information, assigns source page references, generates value-level metadata, and writes the final specification workbook.

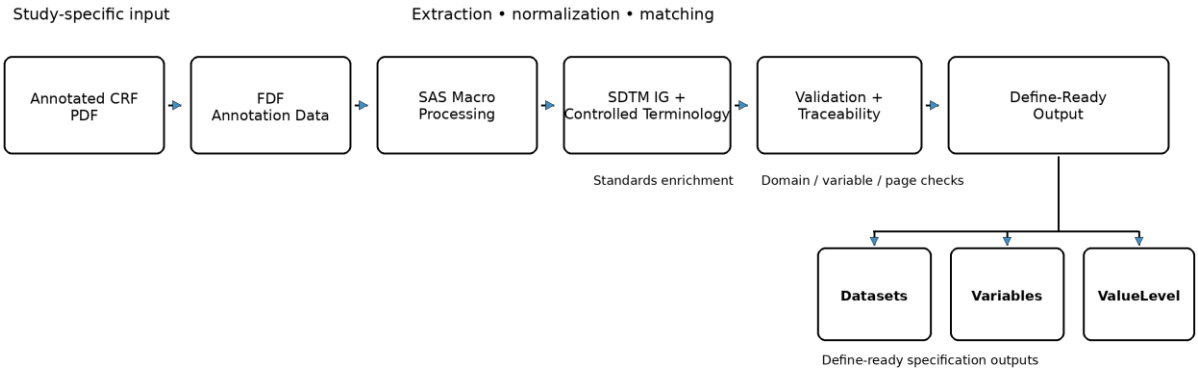


Figure 1. High-Level Architecture of the aCRF-to-Define Automation Framework

The framework is deliberately modular. Each stage creates an intermediate representation that is used by later stages. This makes the processing logic easier to validate and also allows exceptional annotation patterns to be handled without changing the overall output structure.

MACRO IMPLEMENTATION AND SOURCE LISTING

The executable implementation is organized as the %specs macro, with staged processing for reference-data import, FDF parsing, annotation normalization, domain identification, variable generation, Controlled Terminology enrichment, value-level metadata creation, validation, and Excel export. The main macro accepts the SDTM Implementation Guide workbook, Controlled Terminology workbook, output workbook path, and FDF input through macro parameters. The complete source listing used for the framework described in this paper is provided in Appendix A. The example file-system paths in the invocation have been generalized for publication.

```
%macro specs (
    igexcel = ,
    datasetsheet = ,
    varsheet = ,
    ctbook = ,
    ctsheet = ,
    outexcel = ,
    fdf = );

    /* Reference-data import and aCRF/FDF processing */
    /* Domain and variable construction */
    /* Controlled Terminology and value-level enrichment */
    /* Validation and ODS EXCEL export */

%mend specs;
```

INPUTS AND REFERENCE DATA

ANNOTATED CRF FDF

The primary study-specific input is an FDF representation of the annotated CRF. The FDF file is created outside the SAS macro by exporting the annotations from the annotated PDF. The macro consumes the resulting FDF file and treats the annotation objects as the source for domain, variable, value-level, and page information.

The FDF preparation sequence is shown in Figure 2. In Adobe Acrobat, the user opens the annotated CRF, opens the Comments pane, selects the Options menu, chooses Export All To Data File, and saves the output as Acrobat FDF Files (*.fdf) . The resulting file is then supplied to the macro through the FDF= parameter. The Forms Data Format structure consumed by the macro follows the Acrobat/PDF forms data specification.

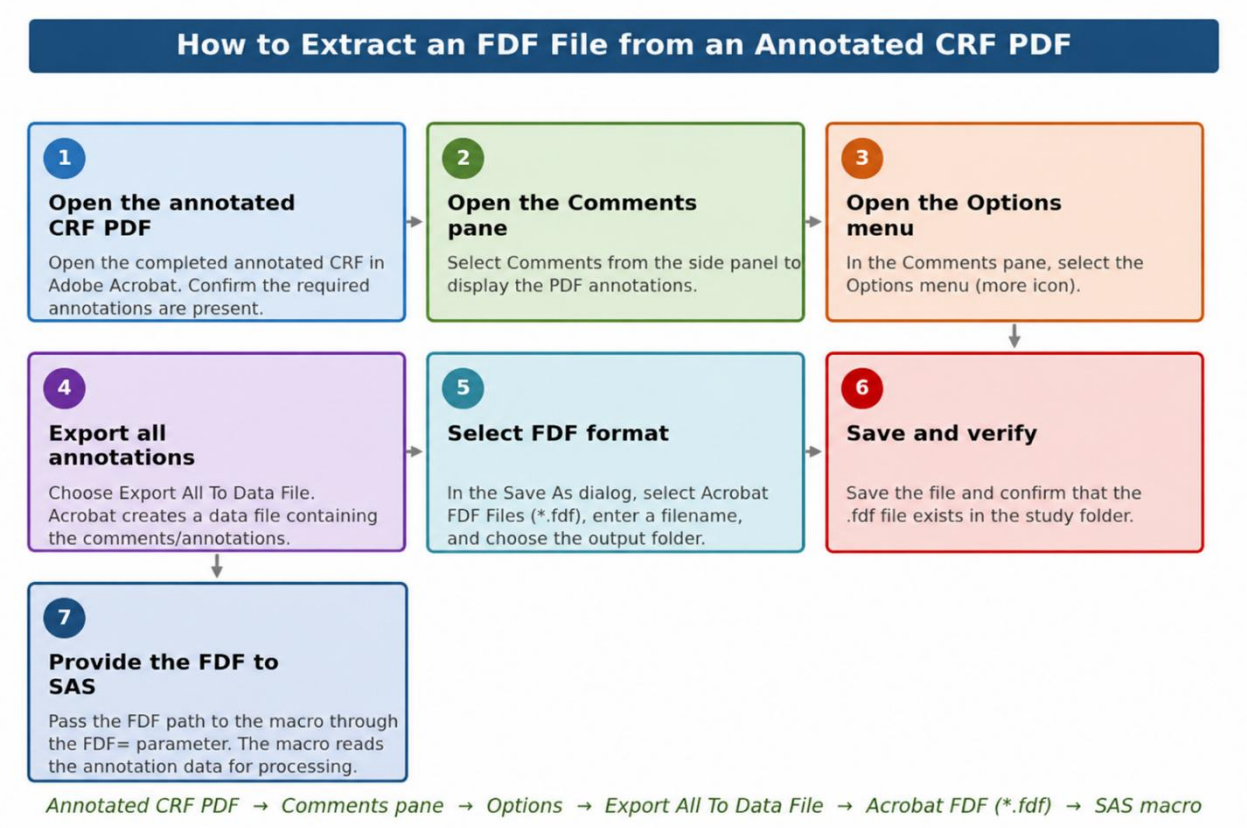


Figure 2. Procedure for Extracting the FDF Input from an Annotated CRF PDF

After receiving the FDF, the macro reads it using a FILENAME statement with encoding='any'. Comment/directive lines beginning with a percent sign are excluded, and the original record sequence is retained so that multi-line PDF annotation objects can be reconstructed.

```
filename fdf "&fdf..fdf" encoding='any';

data fdf_raw;
    infile fdf termstr=cr truncover;
    input line $char32767.;
    if substr(line,1,1)="%" then delete;
    obs+1;
run;
```

SDTM IMPLEMENTATION GUIDE

Two SDTM Implementation Guide worksheets are imported: a dataset-level sheet and a variable-level sheet. Dataset metadata provides the reference domain list and structure information. Variable metadata provides the standard variable names, labels, data types, classes, roles, and Controlled Terminology codelist codes used later in the process.

CONTROLLED TERMINOLOGY

The Controlled Terminology workbook is imported as a reference dataset. The macro first maps codelist codes from the SDTM Implementation Guide variable metadata to readable codelist names. Later, the same terminology source is used to obtain values and labels associated with Test Code and Test Name codelists for value-level metadata.

ANNOTATION EXTRACTION AND NORMALIZATION

A PDF annotation is not necessarily represented by a single physical line in the FDF file. The macro therefore groups FDF records into annotation objects by detecting object boundaries, then transposes the grouped lines so that the complete object can be reconstructed as a single character string.

From each reconstructed annotation object, the macro extracts two key attributes: the annotation text from the /Contents element and the page identifier from the /Page element. The extracted page number is adjusted to the displayed CRF page by adding one to the PDF page index used in the FDF representation.

```
if prxmatch('/\/Contents\((.*?)\)/', line_new) then
    contents = strip(compbl(prxchange(
        's/.*\/Contents\((.*?)\).*/\1/', 1, line_new)));

if prxmatch('/\/Page\s+(\d+)/', line_new) then
    page = strip(compbl(put(input(
        prxchange('s/.*\/Page\s+(\d+).*/\1/', 1, line_new),
        best.)+1,best.)));
```

The macro also recognizes annotations that refer to another page, such as “REFER PAGE” and “SEE PAGE” annotations. It extracts the referenced page numbers and creates duplicated annotation records associated with the target pages. This allows the page information to follow the annotation when the annotation points to a different CRF page.

Before matching annotations to domains or variables, punctuation and special characters are normalized. The contents are converted to a consistent token representation, and a normalized NEWNAME value is generated from the annotation text. This normalization provides a common matching key for subsequent domain and variable identification.

DOMAIN IDENTIFICATION AND VALIDATION

The macro next determines which datasets are represented in the aCRF. Because different domain types require different handling, the annotations are separated into parent domains, supplemental qualifier domains, associated person domains, and custom domains.

PARENT, SUPP, AND ASSOCIATED PERSON DOMAINS

Parent domains are identified when a normalized annotation name directly matches a domain in the SDTM Implementation Guide. SUPP domains are recognized as supplemental qualifier domains even when they do not occur as standalone domain definitions in the imported dataset sheet. Associated Person annotations are detected by the AP prefix and linked back to the corresponding parent domain for metadata construction.

CUSTOM DOMAINS

Custom domains require additional interpretation because they are not directly represented as standard SDTM domains. The macro examines annotation naming patterns to infer the broad model class. Annotations containing TRT are classified as Interventions, annotations containing TERM as Events, and annotations containing TEST as Findings. The detected custom domain is then linked to an appropriate standard model structure to support specification generation.

DOMAIN PAGE CONSOLIDATION AND VALIDATION

Once domains are identified, page numbers are consolidated so that a domain may carry multiple CRF pages in a single comma-separated field. The macro then performs a second validation step by comparing domain candidates derived from annotations with the datasets represented in the SDTM Implementation Guide variable metadata. Mismatches can be separated for review rather than being silently propagated to the final specification.

VARIABLE SPECIFICATION GENERATION

After the domain list has been established, the macro expands the standard SDTM Implementation Guide variable metadata so that it can support AP, SUPP, and custom-domain processing. AP variables are generated by prefixing standard domain variables with AP. SUPPQUAL variables are replicated for the corresponding parent domains. For custom domains, model variables from appropriate standard classes are used as templates.

The expanded variable metadata are then merged with the validated domain list to create the initial Variables specification. The generated structure includes fields such as Dataset, Variable, Label, Data Type, Codelist, Role, Origin, Pages, and Programming Instructions. The macro preserves the Implementation Guide variable order where available and assigns a high order value to records that cannot be assigned a standard order so they can be reviewed separately.

```
data finvars;
    merge expanded_ig_vars(in=b)
          finaldomains(in=a rename=(dataset=dataset_name));
    by dataset_name class;
    if a and b;

    Dataset = Dataset_name;
    Variable = Variable_name;
    Label = variable_label;
run;
```

HANDLING NON-STANDARD OR MODEL VARIABLES

Some annotations represent model or study-specific variables that do not appear directly in the standard variable list. To recognize these annotations, the macro constructs permutations of domain prefixes and model-variable suffixes. An annotation that matches one of these generated names is added as a model variable candidate. This allows the automation to capture a broader range of annotated variables while leaving study-specific interpretation for review.

SOURCE PAGE TRACEABILITY AND ORIGIN

Page assignment is performed after the variable list has been assembled. Annotation records are matched to recognized variable names, and distinct page numbers are retained for each variable. The resulting values are collapsed into a single Pages field. The Origin field is assigned as CRF for annotation-derived records.

The macro applies an additional consistency check by comparing each variable page against the page list for its corresponding dataset. Only matching pages are retained. This step is important when page references have been duplicated through REFER PAGE or SEE PAGE processing because it prevents unrelated annotation pages from being carried into the final specification.

Duplicated annotation records and value-level trigger records with no valid page association are removed during this validation stage. Page values are also intentionally suppressed for SUPP IDVAR variables, which are treated differently from source CRF variables in the generated specification.

CONTROLLED TERMINOLOGY ENRICHMENT

Controlled Terminology enrichment occurs at both the variable and value levels. At the variable level, the macro maps the Controlled Terminology codelist code associated with a standard SDTM variable to the corresponding codelist name. The codelist is then populated in the Variables sheet without requiring the user to manually search the terminology reference.

```
data variables_ig_mod;
    merge forctvarig(in=a)
          forctname(keep=code codelist);
    by code;
    if a;
run;
```

At the value level, the macro identifies terminology records associated with Test Code and Test Name codelists, matches them to VLM variables, and reshapes the terminology data into value-and-label pairs. These pairs are merged back into the VLM records generated from the aCRF annotations.

The combined approach is important because it separates two related but distinct tasks: identifying which codelist applies to a variable and populating the corresponding permissible values and labels when value-level metadata are required.

VALUE-LEVEL METADATA GENERATION

Value-level metadata are generated from annotations that indicate QNAM or TESTCD conditions. SUPP-related annotations are parsed to identify the QNAM and the parent SUPP domain. TESTCD-related annotations are parsed to identify the findings domain and the TESTCD value used to qualify the result variable.

```
if index(dataset,"SUPP") then do;
    variable = "QVAL";
    Where_Clause = "QNAM EQ " || strip(var);
end;
else if length(dataset) eq 2 then do;
    variable = strip(dataset) || "ORRES";
    Where_Clause = strip(dataset) || "TESTCD EQ " || strip(var);
end;
```

The framework creates the appropriate target variable and Where Clause from the annotation context. For supplemental qualifiers, the target variable is QVAL and the condition is based on QNAM. For findings-style value-level metadata, the target variable is the corresponding ORRES variable and the condition is based on TESTCD. Associated Person findings use the parent domain when constructing the target ORRES variable.

The final VLM structure contains fields such as Dataset, Variable, Where Clause, Label, Data Type, Codelist, Origin, Pages, Method, and Developer Notes. Controlled Terminology values and labels are added to the VLM records when a matching Test Code or Test Name terminology definition is available.

REFERENCE VERSIONS AND REPRODUCIBILITY

The automation depends on version-matched reference inputs, so reproducibility requires stating the exact SDTM Implementation Guide and Controlled Terminology release used. Table 1 summarizes the reference versions used for the framework application described in this paper.

Reference	Published baseline	Notes
SDTM Implementation Guide	SDTMIG v3.4	Reference standard used for the pilot reported in this paper.
Controlled Terminology	2026-03-27 release	Reference terminology used for the pilot reported in this paper.
Define-XML	v2.1.11 (6 Apr 2026)	Target standard for downstream Define-XML generation.
FDF extraction workflow	Adobe Acrobat export to FDF	Annotations exported using Acrobat's Comments > Export All To Data File function.

Table 1. Reference Standards and Versions Used for the Framework Application Described in This Paper

OUTPUT SPECIFICATION AND FORMATTING

The final workbook contains three output sheets — Datasets, Variables, and ValueLevel — structured to support downstream Define-XML development using the Define-XML v2.1.11 metadata model. These are generated from the final domain, variable, and VLM datasets produced during the macro run. A reusable export macro dynamically determines character-variable widths, assigns display labels, imposes a consistent column order, and writes the results using ODS EXCEL.

```
ods excel file="&outexcel..xlsx";

%export_sheet(
    ds=finaldomains,
    sheet=Datasets,
    labeltype=DATASETS
);

%export_sheet(
    ds=final_spec,
    sheet=Variables,
    labeltype=VARIABLES
);

%export_sheet(
    ds=vlm_final,
    sheet=ValueLevel,
    labeltype=VLM
);
```

The export routine also applies color-coded review flags during the same PROC REPORT step used to build each worksheet. These flags do not replace validation; they direct reviewer attention to specific categories of Permissible variables that require spec-writer judgment before the specification is finalized.

On the Variables sheet records are highlighted in yellow when Permissible variables are annotated in the aCRF but are not present in the SDTM Implementation Guide — typically model or study-specific variables recognized through the annotation permutation matching described earlier. The yellow flag signals that the specification writer needs to review these variables and arrange them in the appropriate order within the dataset variable sequence.

On the Variables sheet, an additional flag highlights PERM (Permissible) variables in light red when the corresponding Pages field is blank, indicating that the variable is present in the SDTM Implementation Guide for the domain but was not annotated on the aCRF. Because Permissible variables are optional, this flag directs the specification writer to review each one and decide whether to retain it, for example when it will be populated through a derivation, or remove it as not applicable to the study.

Table 2 summarizes the metadata components generated automatically by the framework and the primary source or processing logic used for each component.

Component	Primary Source / Logic	Generated Output
Datasets	aCRF annotations + SDTM IG	Domain metadata, structure, key variables, class, and CRF pages
Variables	aCRF annotations + SDTM IG	Dataset, variable, label, type, role, codelist, origin, pages, and related metadata fields
Codelists	SDTM IG + Controlled Terminology	Applicable codelist for standard variables
Value-Level Metadata	aCRF QNAM/TESTCD annotations + Controlled Terminology	Where Clause, value, label, origin, and CRF page information
Review Flags	Validation and export logic	Color-coded flags: yellow for order placement, light red for keep/drop review

Table 2. Automated Metadata Components Produced by the Framework

ILLUSTRATIVE END-TO-END EXAMPLE

Consider an annotated CRF page containing a standard domain annotation, a variable annotation, and a TESTCD condition. The FDF contains the corresponding annotation objects and page identifiers. The macro reconstructs the annotation objects, extracts the contents and page, normalizes the annotation text, and matches the resulting names to SDTM Implementation Guide metadata.

The domain is added to the Datasets structure, the standard variables are carried into the Variables specification, and the annotated page is retained in the Pages field. When the variable has an applicable Controlled Terminology codelist, the codelist name is populated automatically. When the annotation identifies a value-level condition, the record is routed to the VLM output and the appropriate Where Clause is generated.

For TESTCD-driven value-level metadata, the matching Controlled Terminology entries are used to populate the controlled value and label information. The resulting Datasets, Variables, and ValueLevel records therefore contain both information extracted from the aCRF and standards-based information derived from the reference sources.

Figure 3 illustrates this flow using generic example values from the LB (Laboratory Test Results) domain, showing the Dataset-, Variable-, and Value-level outputs produced from a single annotated CRF page.

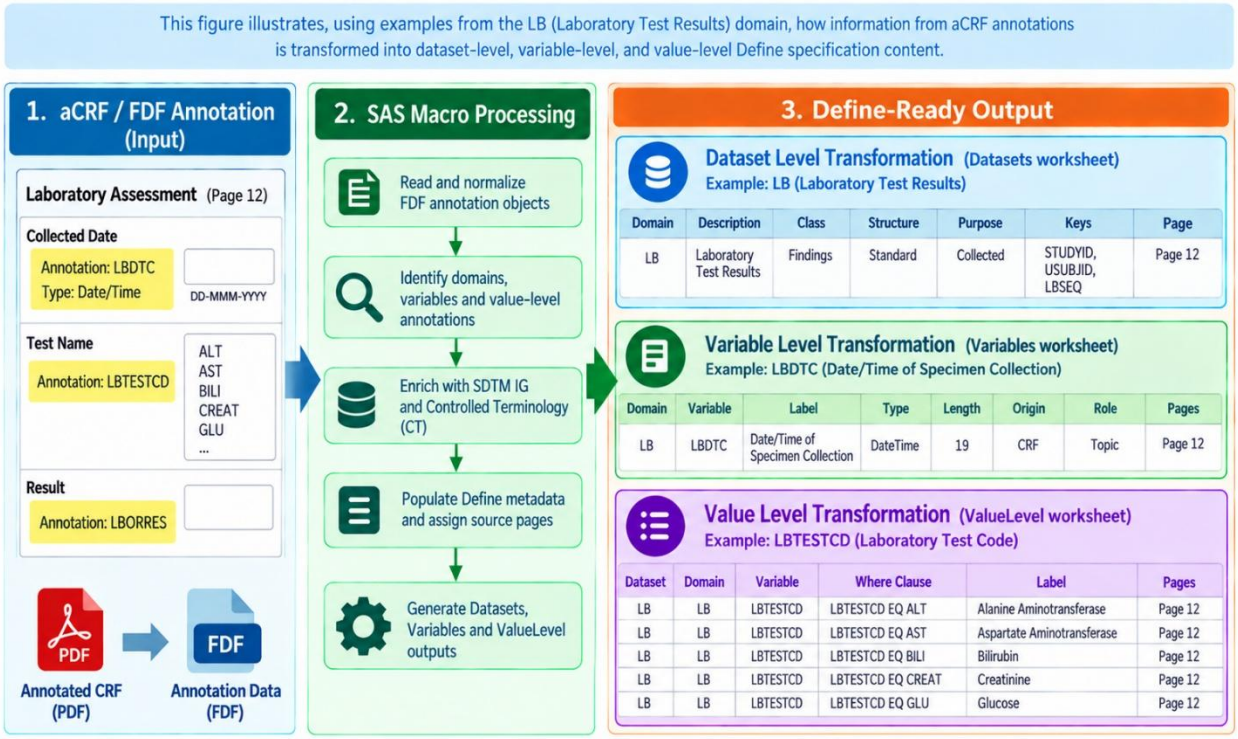


Figure 3. Illustrative Transformation from aCRF Annotation to Define-Ready Specification

FRAMEWORK APPLICATION AND PRACTICAL CONSIDERATIONS

The framework was applied using an annotated CRF together with the corresponding SDTM Implementation Guide and Controlled Terminology reference data. The application demonstrates how the automated workflow can support the repeatable first-pass activities normally performed during Define specification preparation.

The generated specification structures cover the repeatable transformation of source information, while manual review remains necessary for study-specific derived variable specifications, confirmation of value-level metadata groupings and CRF page references, and other decisions that require study context.

The main practical opportunity for automation is the repeatable transformation of already-available source information: reading annotations, identifying domains and variables, carrying source pages, finding standard codelists, and constructing repeatable value-level structures. Manual review remains necessary for study-specific interpretation and for resolving cases where the source annotations do not provide enough information to make a reliable specification decision.

PILOT RESULTS AND PERFORMANCE EVALUATION

To assess the practical impact of the framework, first-pass specification preparation time can be compared between the conventional manual approach and the automated workflow. The comparison should be based on the same annotated CRF and the same specification scope for both approaches.

The time comparison reflects first-pass specification preparation. Final programmer and standards review remains necessary for study-specific derivations, ambiguous annotations, value-level grouping decisions, and other content that cannot be reliably inferred from the source annotations.

A pilot time comparison was performed on the same annotated CRF (*250 pages, 36 domains, 750 variables*) using the same specification scope for both approaches: Datasets, Variables, and ValueLevel sheets. Manual first-pass preparation required 24–32 hours. The automated workflow required approximately 1 hour, including FDF export, preparation of the IG and CT workbooks, macro execution, and loading of the output. This represents a 96–97% reduction in first-pass preparation time (24–32x faster). The comparison reflects first-pass preparation only; final programmer and standards review, which remains necessary for study-specific derivations, ambiguous annotations, and value-level grouping decisions, is not included.

Measure	Manual Approach (Actual)	Automated Workflow (Actual)
First-pass specification preparation time	24-32 hours	1 hour. This includes FDF export, preparing the IG and CT workbooks, the macro run time, and loading the output
Time reduction/Efficiency	—	96–97% reduction in preparation time (24–32× faster)

Table 3. First-Pass Specification Preparation Time Comparison

VALIDATION AND REVIEW CONSIDERATIONS

The framework includes several internal validation mechanisms. Domain candidates are checked against the variable metadata available from the SDTM Implementation Guide. Variable page numbers are validated against the pages assigned to their parent domains. Duplicate annotation records are removed, and records that cannot be associated with a valid domain or source page are excluded or flagged for review.

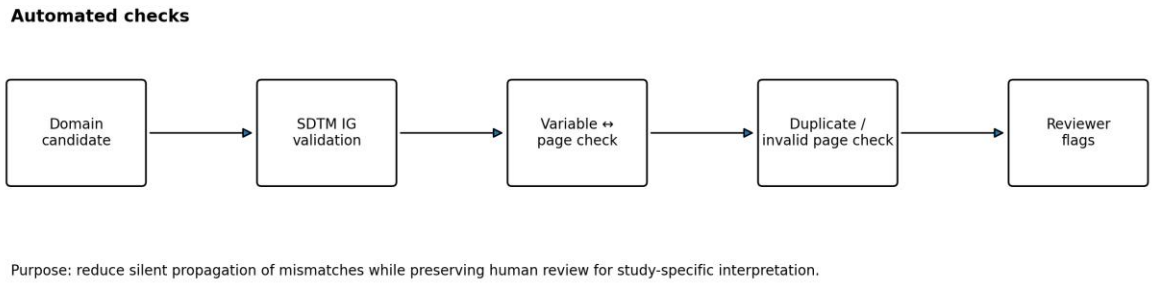


Figure 4 summarizes the internal validation and review gates applied before the generated metadata are exported to the specification workbook.

Figure 4. Automated Validation and Review Gates

Controlled Terminology matching also introduces a dependency on the reference terminology content provided to the macro. The codelist mapping and TESTCD/Test Name value enrichment are therefore only as reliable as the reference workbook and its alignment with the study standards. The macro should be executed with the intended SDTM Implementation Guide and Controlled Terminology versions used for the study.

The resulting workbook is an initial specification draft. Programmer and standards review remains essential for study-specific derived variables, complex logic, assumptions, metadata groupings, and any

ambiguous annotation. The automation is intended to move effort away from transcription and toward interpretation and quality review.

BENEFITS AND LIMITATIONS

BENEFITS

The framework provides a repeatable method for turning aCRF annotations into specification-ready metadata. It reduces repeated transcription, carries CRF page traceability into the specification, automatically applies standard codelists where available, and produces value-level metadata with controlled values and labels. The separation of extraction, validation, enrichment, and export also provides a transparent processing path that can be reviewed and tested.

An additional benefit is explicit support for annotation patterns that frequently complicate manual specification development. The macro handles referenced-page annotations, SUPP and AP structures, custom domains, QNAM conditions, TESTCD conditions, and model-variable candidates rather than treating all annotations as simple one-to-one variable mappings.

LIMITATIONS

The framework relies on consistent annotation conventions. If annotations are incomplete, ambiguous, inconsistently worded, or missing expected domain/variable information, the output may require more manual correction. Custom domain classification and model-variable recognition are based on implemented pattern rules and should therefore be reviewed when a study uses unusual annotation conventions.

The current implementation produces a Define specification-ready Excel structure rather than directly generating a final Define-XML file. Final metadata approval, study-specific derivations, and other content that cannot be reliably inferred from the aCRF or reference datasets remain outside the scope of the automated generation step.

CONCLUSION

This paper presents a SAS® macro-based framework that automates the transformation of annotated CRF information into an initial Define specification draft. The framework combines FDF annotation processing with SDTM Implementation Guide metadata and Controlled Terminology to generate dataset-, variable-, and value-level specification content in a consistent structure that supports downstream Define-XML development.

The main contribution is the integration of annotation extraction, validation, page traceability, and standards-based enrichment within a single repeatable workflow. Rather than simply extracting annotations, the framework carries information from the aCRF into the specification and augments it with codelist and value information from Controlled Terminology, providing a traceable starting point for programming and review while leaving study-specific interpretation to the reviewer.

The framework provides a repeatable approach for reducing repetitive specification-preparation activities while preserving human review for study-specific interpretation and final specification approval. Future work can evaluate the framework across additional studies, characterize post-generation corrections, and expand the framework to cover additional study-specific metadata patterns while preserving appropriate human review.

APPENDIX A. COMPLETE SAS MACRO SOURCE LISTING

The following listing is the complete SAS program supplied with the framework. It is included to document the implementation described in the paper and to support reproducibility. The executable logic is retained from the supplied source; only the example file-system paths have been generalized for publication.

```
/*====================================================================
PROGRAM: Automated Define Specification Generator from Annotated CRF
AUTHOR: Bharath Pappula
PURPOSE:
This program reads an annotated CRF FDF file and combines the
annotation information with SDTM IG metadata and Controlled Terminology
to generate an initial Define specification workbook.

MAIN OUTPUTS:
1. Datasets   - detected domains with labels, structures and CRF pages
2. Variables  - detected/model variables with CT codelists and pages
3. ValueLevel - value-level metadata with TEST/QNAM conditions,
CT values/labels and CRF page traceability

NOTE:
Comments have been added to describe the logical processing steps.
The underlying processing logic is retained from the original macro.
====================================================================*/

dm "log" clear;

%macro specs (
    igexcel = ,
    datasetsheet = ,
    varsheet = ,
    ctbook =  ,
    ctsheet =  ,
    outexcel = ,
    fdf = );

    /*====================================================================
    STEP 0: INITIALIZE THE WORK LIBRARY
    --------------------------------------------------------------------
    Remove temporary WORK datasets from previous runs so that the macro
    always starts from a clean state.
    ====================================================================*/
    proc datasets lib=work kill nolist;
    quit;


    /*====================================================================
    STEP 1: CREATE A UTILITY MACRO TO RETURN DATASET OBSERVATION COUNTS
    --------------------------------------------------------------------
    GET_NOBS checks whether a dataset exists and returns its observation
    count in a global macro variable.
    ====================================================================*/
    %macro get_nobs(ds,mvar);

        %global &mvar;

        %if %sysfunc(exist(&ds)) %then %do;
            %let dsid=%sysfunc(open(&ds));
            %let &mvar=%sysfunc(attrn(&dsid,NLOBS));
            %let rc=%sysfunc(close(&dsid));
        %end;
        %else %let &mvar=0;

    %mend;


    /*====================================================================
    STEP 2: IMPORT REFERENCE METADATA
    --------------------------------------------------------------------
    Three reference inputs are required:

    - SDTM IG Datasets sheet
    - SDTM IG Variables sheet
    - Controlled Terminology workbook/sheet

    These reference workbooks provide the standard domain definitions,
    standard variable definitions and codelist information
    ====================================================================*/

    /*---------------------------- SDTM IG --------------------------------*/
```

```
PROC IMPORT OUT= WORK.domains_IG(rename=( dataset_name = dataset_merge))
    DATAFILE= "&igexcel."
    DBMS=EXCEL REPLACE;
    RANGE="&datasetsheet.$";
    GETNAMES=YES;
    MIXED=NO;
    SCANTEXT=YES;
    USEDATE=YES;
run;

/* Sort domains for subsequent BY-group MERGE processing. */
proc sort data = domains_IG ;
    by dataset_merge;
run;

/* Import standard variable-level metadata. */
PROC IMPORT OUT= WORK.variables_IG
    DATAFILE= "&igexcel."
    DBMS=EXCEL REPLACE;
    RANGE="&varsheet.$";
    GETNAMES=YES;
    MIXED=NO;
    SCANTEXT=YES;
    USEDATE=YES;
run;

/* Sort variables for later matching. */
proc sort data = variables_IG;
    by variable_name;
run;

/*---------------------- Controlled Terminology -----------------------*/

/* Import CT data used for codelist and value/label enrichment. */
PROC IMPORT OUT= WORK.ct
    DATAFILE= "&ctbook."
    DBMS=EXCEL REPLACE;
    RANGE="&ctsheet.$";
    GETNAMES=YES;
    MIXED=NO;
    SCANTEXT=YES;
    USEDATE=YES;
run;


/*=====================================================================
STEP 3: ADD READABLE CODELIST NAMES TO STANDARD VARIABLES
---------------------------------------------------------------------
The SDTM IG variable metadata contains CT codelist codes. The CT
workbook contains the corresponding submission values/names.

Merge these two sources so the Variables sheet can contain
codelist name rather  CT code. This is done to match the define specification format
=====================================================================*/

/* Keep CT records representing codelists and rename the submission
value to the Codelist field used in the specification. */
proc sort data = ct out=forctname(rename=(CDISC_Submission_Value=codelist));
    by code;
    where Codelist_Extensible__Yes_No_ ne "";
run;

/* Rename the IG CT codelist code to the merge key CODE. */
proc sort data = variables_ig out=forctvarig(rename=(CDISC_CT_Codelist_Code_s_=code));
    by CDISC_CT_Codelist_Code_s_;
run;

/* Each standard variable with its corresponding codelist name. */
data variables_ig_mod;
    merge forctvarig(in=a) forctname(keep=code codelist)   ;
    by code;
    if a;
          order = input(variable_order,best.);
run;

/*---------------- End of IG and CT preparation -----------------------*/


/*=====================================================================
STEP 4: READ THE PDF FDF ANNOTATION FILE
```

```
    ----------------------------------------------------------------------
    The FDF file contains the annotations created in the annotated CRF.
    Each line is read as character text. Comment/directive lines beginning
    with '%' are excluded, and OBS preserves the original sequence.
    ======================================================================*/
    filename fdf "&fdf..fdf"  encoding='any';

    data fdf_raw;
        infile fdf termstr=cr truncover;
        input line $char32767.;
        if substr(line,1,1)="%" then delete;
        obs+1;
    run;

    /* Preserve original annotation order. */
    proc sort data  = fdf_raw ;
        by obs;
    run;


    /*======================================================================
    STEP 5: GROUP FDF LINES INTO INDIVIDUAL ANNOTATION OBJECTS
    ----------------------------------------------------------------------
    A PDF annotation may span multiple FDF records. OBJ is used as the
    start of an annotation object and ENDOBJ marks its end.

    OBJ_NUM groups the lines belonging to the same annotation.
    ======================================================================*/
    data want;
        set fdf_raw;
        retain obj_num 0;

        if findw(upcase(line), 'OBJ') and
        findw(upcase(line), 'ENDOBJ') = 0 then do;
            obj_num + 1;
        end;
    proc sort;
        by obj_num;
    run;


    /*======================================================================
    STEP 6: RECONSTRUCT EACH ANNOTATION AND EXTRACT CONTENT/PAGE
    ----------------------------------------------------------------------
    For each annotation object:

    - Rebuild all lines into LINE_NEW.
    - Extract the /Contents(...) annotation text.
    - Extract /Page and convert the internal PDF page number to the
    displayed page number by adding 1.
    - Detect "REFER PAGE" / "SEE PAGE" annotations.
    - Extract referenced page numbers into PAGEREF.
    ======================================================================*/
    proc transpose data = want out= want1;
        by obj_num;
        var line;
    run;

    data new(where=(not missing(contents)));
        set want1;

        length contents $32000 page $20 line_new $32000;

        /* Rebuild complete FDF object text. */
        line_new = compbl(catx(' ', of col:));

        /* Extract annotation contents. */
        if prxmatch('/\/Contents\((.*?)\)/', line_new) then
        contents = strip(compbl(prxchange('s/.*\/Contents\((.*?)\).*/\1/', 1, line_new)));

        /* Extract page  */
        if prxmatch('/\/Page\s+(\d+)/', line_new) then
        page = strip(compbl(put(input(prxchange('s/.*\/Page\s+(\d+).*/\1/', 1, line_new),best.)+1,best.)));

        /* Detect page-reference annotations and extract all referenced pages. This is done as preliminary
step to
                                replicate annotations in to their target pages ("eg: Refer Page xx for
Annotations"*/
        if (index(upcase(Contents),"REFER") and index(upcase(Contents),"PAGE")) or
        (index(upcase(Contents),"SEE") and index(upcase(Contents),"PAGE"))  then do;
```

```
        array num[100] $20 numx1-numx100;
        length token $100 number $100;
        call missing(of num1-num100);

        idx = 1;

        do i = 1 to countw(contents, ' ,');
        token = scan(contents, i, ' ,');

        /* Keep digits only so text such as "page 12" yields 12. */
        number = compress(token, , 'kd');

        if not missing(number) then do;
            if idx <= dim(num) then do;
                num[idx] = number;
                idx + 1;
            end;
        end;
    end;

    pageref = catx(",",of numx:);

    end;

run;


/*=====================================================================
STEP 7: EXPAND MULTIPLE "REFER PAGE" TARGETS
---------------------------------------------------------------------
A single annotation can refer to multiple CRF pages. Convert the
comma-separated page references into one record per reference so the
source annotation can later be copied to each target page.
=====================================================================*/
data ref_expanded(where=(not missing(refpage)));
    set new;
    obs = _n_;
    length refpage 8;
    targetpage = page;

    do i = 1 to countw(pageref, ',');
    refpage = input(scan(pageref, i, ','), best.);
    output;
    end;

    keep targetpage refpage;
run;


/*=====================================================================
STEP 8: DUPLICATE REFERENCED ANNOTATIONS TO TARGET PAGES
---------------------------------------------------------------------
The EXP macro creates a copy of an annotation on the page to which it
refers. The PAGE value embedded inside LINE_NEW is also updated.
=====================================================================*/
%macro exp (sourcepg= ,targetpg=);

    data expanded_&targetpg;
        set new;
        where page = "&sourcepg";

        /* Assign target page to duplicated annotation. */
        page = "&targetpg.";

        /* Replace the embedded PDF page reference as well. */
        line_new = prxchange(
        's/\/Page\s*\d+/\/Page ' || strip("&targetpg") || '/',
        -1,
        line_new
        );

    run;

%mend exp;

%get_nobs(ref_expanded,exp_nobs);
  %if &exp_nobs > 0 %then %do;

   /* Dynamically execute EXP for every source/target page pair. */
  data _null_;
      set ref_expanded;
```

```
    exec = "%" || "exp" || "(" ||
    "sourcepg=" || strip(put(refpage,best.)) ||
    ",targetpg=" ||  strip(targetpage) || ")";
    call execute(exec);
run;

/* Combine duplicated annotations and the original annotations. */
data expanded;
    set expanded_:;
run;

data new_expanded;
    set new expanded;
run;
 %end;
 %else %do;
data new_expanded;
    set new  ;
run;
 %end;

/*=========================================================================
STEP 9: NORMALIZE ANNOTATION TEXT
-------------------------------------------------------------------------
Annotation contents can contain special/ PDF-specific
characters. Remove/replace these characters so domain and variable
identifiers can be extracted consistently.

NEWNAME contains normalized uppercase annotations.
=========================================================================*/
data anno(keep=newname contents page line_new);
    length newname $32000;
    set new_expanded;

    contents = compress(contents, "'","");
    contents = translate(contents," ",".");
    contents = translate(contents," ",",");
    contents = translate(contents," ","/");
    contents = translate(contents," ",":");
    contents = translate(contents," ",";");
    contents = translate(contents," ","(");
    contents = translate(contents," ",")");
    contents = translate(contents," ","=");
    contents = translate(contents," ","|");
    contents = translate(contents," ","\");

    /* Create one normalized record per word in the annotation. */
    do i=1 by 1 while (scan(contents,i," ") ne " ");
    newname= compress(strip(upcase(strip(scan(contents,i," ")))),"","kn");
    output;
    end;
run;


/*=========================================================================
STEP 10: IDENTIFY DOMAIN ANNOTATIONS
-------------------------------------------------------------------------
APxx annotations represent Associated Persons domains. For matching
to the standard domain list, the AP prefix is removed and ORIG=AP is
stored.
=========================================================================*/
data dom_check;
    set anno;

    if substr(upcase(newname),1,2) = "AP" and length(newname) = 4 then do;
        dataset_merge = strip(substr(newname,3));
        orig = "AP";
    end;
    else do;
        dataset_merge  = strip(newname);
    end;
proc sort ;
    by dataset_merge;
run;


/*=========================================================================
STEP 11: SEGREGATE DOMAIN TYPES
-------------------------------------------------------------------------
Domain annotations are divided into:
- Parent standard domains
```

```
    - SUPP domains
    - Associated Persons domains
    - Custom X/Y/Z domains
    ======================================================================*/
    data    parent_domain supp_domain ap_domain custom_domain ;
        merge  dom_check(in=b) domains_IG(in=a);
        by dataset_merge;

        if a and b and newname eq dataset_merge then output parent_domain;
        if b and not a and index(upcase(dataset_merge),"SUPP") and length(dataset_merge) eq 6 then output
supp_domain;
        if a and b and orig = "AP" then output ap_domain;
        if substr(upcase(dataset_merge),1,1) in ("X","Y","Z") and index(upcase(dataset_merge),"SUPP") eq 0
then output custom_domain;
    run;


    /*======================================================================
    STEP 12: PROCESS CUSTOM DOMAINS
    ----------------------------------------------------------------------
    For X/Y/Z custom domains, infer the intended model class using
    annotation naming patterns:
    - TRT / TRT-related names -> Interventions
    - TERM / TERM-related names -> Events
    - TEST / ORRES-related names -> Findings

    The inferred class is then mapped to an appropriate standard model
    domain (VS, CM or DS) for building the specification template.
    ======================================================================*/
    %let cnts=0;

    proc sql;
        select distinct strip(dataset_merge)
        into :doms separated by ","
        from custom_domain
        where not missing(dataset_merge) and length(strip(dataset_merge)) eq 2;

        select distinct count(dataset_merge) into:cnts
        from custom_domain
        where not missing(dataset_merge) and length(strip(dataset_merge)) eq 2;
    quit;

    %if &cnts gt 0 %then %do;

        %put &doms.;

        data cust ;
            set anno;
            length value $1000;

            do i = 1 to &cnts.;
            value = scan("&doms.", i, ',');

            if index(newname, strip(value)) then do;
                if index(strip(newname),"TRT") then class= "Interventions";
                if index(strip(newname),"TERM") then class= "Events";
                if index(strip(newname),"TEST") or index(strip(value),"ORRES") then class= "Findings";
                output;
            end;
            end;
        run;

        data cust_class;
            set cust;
            dataset_merge = strip(substr(newname,1,2));
        proc sort;
            by dataset_merge ;
            where not missing(class);
        run;

        /* Build a custom domain label from the annotation contents. */
        data cust_dom;
            set cust;
            dataset_merge = strip(substr(newname,1,2));

            length cust_label $200;
            cust_label = "";

            i = 2;
            do while(scan(compbl(contents), i, ' ') ne "");
            cust_label = catx(' ', cust_label, scan(compbl(contents), i, ' '));
```

```
        i + 1;
        end;
    run;

    proc sort nodupkey;
        by dataset_merge ;
        where length(newname) eq 2;
    run;

    /* Map custom model class to the standard structure used as a template. */
    data custom_domain1;
        merge cust_dom cust_class;
        by dataset_merge;

        cust_dom =  dataset_merge;
        cust_label = cust_label;
        if class = "Findings" then do; dataset_merge = "VS"; end;
        if class = "Interventions" then do; dataset_merge = "CM"; end;
        if class = "Events" then do; dataset_merge = "DS"; end;
    proc sort;
        by dataset_merge;
    run;

    data custom_domain_final;
        merge custom_domain1(in=a) domains_ig(in=b);
        by dataset_merge;
        if a ;
    run;
%end;


/*=====================================================================
STEP 13: COLLAPSE DOMAIN PAGE NUMBERS
---------------------------------------------------------------------
A domain can be annotated on multiple CRF pages. Transpose the unique
page values and combine them into a single comma-separated PAGE field.
This is performed separately for parent, SUPP and AP domains.
=====================================================================*/

/* Parent-domain pages. */
proc sort data = parent_domain nodupkey;
    by dataset_merge dataset_label page;
run;

proc transpose data =  parent_domain out=parent_page;
    by dataset_merge dataset_label;
    var page;
run;

data parent_domain_final;
    merge parent_domain(drop=page) parent_page;
    by dataset_merge dataset_label;
    page = catx(",",of col:);
run;

/* SUPP-domain pages. */
proc sort data = supp_domain nodupkey;
    by dataset_merge page;
run;

proc transpose data =  supp_domain out=supp_page;
    by dataset_merge ;
    var page;
run;

data supp_domain_final;
    merge supp_domain(drop=page) supp_page;
    by dataset_merge ;
    page = catx(",",of col:);
run;

/* AP-domain pages, only when AP annotations exist. */
%get_nobs(ap_domain,ap_nobs);
%if &ap_nobs > 0 %then %do;
    proc sort data = ap_domain ;
        by dataset_merge dataset_label;
    run;

    proc transpose data =  ap_domain out=ap_page;
        by dataset_merge dataset_label;
        var page;
```

```
        run;

        data ap_domain_final;
            merge ap_domain(drop=page) ap_page;
            by dataset_merge dataset_label;
            page = catx(",",of col:);
        run;
    %end;


    /*=====================================================================
    STEP 14: DETERMINE WHICH DOMAIN GROUPS EXIST
    -----------------------------------------------------------------------
    Observation counts control conditional SET statements in the final
    domain-building step.
    =====================================================================*/
    %get_nobs(parent_domain_final,parent_nobs);
    %get_nobs(supp_domain_final,supp_nobs);
    %get_nobs(ap_domain_final,ap_nobs);
    %get_nobs(custom_domain_final,cust_nobs);

    %put &=parent_nobs &=supp_nobs &=ap_nobs &=cust_nobs;


    /*=====================================================================
    STEP 15: BUILD THE DATASETS SPECIFICATION
    -----------------------------------------------------------------------
    Combine detected domains with standard IG metadata and populate the
    Datasets-sheet fields.

    Special handling is applied to:
    - SUPP domains
    - standard/trial domains
    - custom domains
    - Associated Persons domains
    =====================================================================*/
    data domains;
        length dataset label class SubClass structure Key_Variables standard Has_No_Data Repeating
Reference_Data comment Developer_Notes page structure_new   $1000;
        retain standard;
        call missing(structure_new,temp);

        set %if &parent_nobs > 0 %then %do; parent_domain_final  (in= parent)  %end;
            %if &supp_nobs > 0 %then %do; supp_domain_final (in=supp) %end;
                %if &ap_nobs > 0 %then %do; ap_domain_final (in=ap) %end;
                    %if &cust_nobs > 0 %then %do; custom_domain_final (in=cust) %end;
  domains_ig (where=(index(upcase(dataset_label),"TRIAL") and dataset_merge not in ("TM","TD")) in=td)
  domains_ig (where=(upcase(dataset_merge)="SE" ) in=se) ;

  %if &supp_nobs > 0 %then %do;
      if supp then do;
          Dataset = dataset_merge;
          Label = "Supplemental Qualifiers for " || substr(dataset_merge,5);
          Class = "Relationship";
          Structure = "One record per IDVAR, IDVARVAL, and QNAM value per subject";
          Key_Variables = "STUDYID, RDOMAIN, USUBJID, IDVAR, IDVARVAL, QNAM";
      end;
  %end;

  /* Standard/Trial/Special-purpose domains inherit IG metadata. */
  if parent or td or se then do;
      Dataset = dataset_merge;
      Label = dataset_label;
      Class = class;
      Structure = Structure;
      Key_Variables = " ";
  end;

  %if &cust_nobs > 0 %then %do;
      if cust then do;
          Dataset = cust_dom;
          Label = cust_label;
          Class = class;
          Structure = Structure;
          Key_Variables = "";
      end;
  %end;

  /* Transform the IG structure text for Associated Persons domains. */
  %if &ap_nobs > 0 %then %do;
      if ap then do;
```

```
          Dataset = newname;
          Label = "Associated Persons " || strip(dataset_label);
          Class = class;
          temp =  length((strip(scan(structure,1," ")) || " " || strip(scan(structure,2," ")))) +1;
          structure_new = catx(' ', "One record per Associated person", strip(substr(structure,temp)) );
          Key_Variables = " ";
      end;
  %end;

  Standard = version;
     call missing(
         Has_No_Data,
         Repeating,
         Reference_Data,
         Comment,
         Developer_Notes,
         SubClass
     );

  proc sort nodupkey;
      by Dataset;
  run;


  /*=========================================================================
  STEP 16: FORMAT THE FINAL DATASETS METADATA
  -------------------------------------------------------------------------
  Apply the user-facing column labels and retain only the Datasets sheet
  columns required for the specification output.
  =========================================================================*/
  data domains_fin;
      set domains;
      if not missing(structure_new) then structure = structure_new;
      else structure= structure;
      label
      dataset         = "Dataset"
      label           = "Label"
      class           = "Class"
      SubClass        = "SubClass"
      structure       = "Structure"
      Key_Variables   = "Key Variables"
      standard        = "Standard"
      Has_No_Data     = "Has No Data"
      Repeating       = "Repeating"
      Reference_Data  = "Reference Data"
      comment         = "Comment"
      Developer_Notes = "Developer Notes"
      page            = "Page";
      keep dataset label class SubClass structure Key_Variables standard Has_No_Data Repeating
Reference_Data comment Developer_Notes page;
  run;


  /*=========================================================================
  STEP 17: REVALIDATE DETECTED DOMAINS
  -------------------------------------------------------------------------
  Compare annotation-derived domain names against domains represented in
  the IG variable metadata. Mismatches are separated into REVIEW so they
  can be examined rather than silently becoming false datasets.
  =========================================================================*/
  proc sql noprint;
      select quote(strip(newname)) into: varnames separated by "," from anno
      where length(newname) gt 4 and length(newname) le 8 and index(upcase(newname),"VISIT") eq 0 ;
  quit;

  data check_exist_domains;
      set variables_ig_mod(where =(Dataset_Name ne "SUPPQUAL"));
      if variable_name in (&varnames.);
  proc sort nodupkey;
      by dataset_name;
  run;

  proc sort data = domains_fin;
      by dataset;
  run;

  data finaldomains review;
      merge domains_fin(in=a) check_exist_domains(keep= dataset_name in=b rename=(dataset_name=dataset));
      by dataset;

      if (( (a and not b) or (b and not a) ) and
```

```
      (substr(dataset,1,4) ne "SUPP" and substr(dataset,1,2) ne "AP" and substr(dataset,1,1) not in
("X","Y","Z","T")
      and upcase(dataset) not in ("SE","RELREC"))) then do;
          output review;
      end;
      else do;
          output finaldomains;
      end;
  run;


  /*=================================================================
  STEP 18: CREATE MODEL RECORDS FOR AP AND SUPP DOMAINS
  -------------------------------------------------------------------
  Use standard IG variables as templates so AP and SUPP domains have the
  appropriate standard/model variables available for annotation matching.
  =================================================================*/

  /* Duplicate each standard variable record as an AP version. */
  data variables_ig_expanded_AP;
      set variables_ig_mod;
      output;
      dataset_name = cats('AP',dataset_name);
      output;
      variable_name = "RSUBJID";
      variable_label = "Related Subject";
       order = 3.1;
      output;
      variable_name = "APID";
      variable_label = "Associated Persons Identifier";
       order = 3.2;
      output;
      variable_name = "SREL";
      variable_label = "Subject, Device, or Study Relationship";
       order = 3.3;
      output;
run;
proc sort data =  variables_ig_expanded_AP nodupkey;
by dataset_name variable_name ;
run;

  /* Identify parent domains for creation of SUPPxx templates. */
  proc sql;
      select distinct dataset_name
      into :domains separated by ' '
      from variables_ig_expanded_AP
      where length(dataset_name) le 4;
  quit;

  /* Create SUPP<parent> variable templates. */
  data suppdummy_main;
      set variables_ig_mod(where=(upcase(dataset_name)='SUPPQUAL'));
      do i = 1 to countw("&domains", ' ');
      parent = scan("&domains", i, ' ');
      dataset_name = cats('SUPP', parent);
      output;
      end;
  run;
  proc sort;
      by dataset_name order;
  run;


  /*=================================================================
  STEP 19: CREATE MODEL TEMPLATES FOR CUSTOM X/Y/Z DOMAINS
  -------------------------------------------------------------------
  When custom X/Y/Z domains exist, use relevant standard model domains
  as templates. Domain-specific prefixes are replaced so standard model
  variables can be mapped into the custom domain context.
  =================================================================*/
  proc sql noprint;
      select distinct dataset
      into :domains separated by ' '
      from finaldomains
      where length(dataset) eq 2 and substr(dataset,1,1) in ("X","Y","Z");

      select count(distinct dataset)
      into :custcnt
      from finaldomains
      where length(dataset) eq 2 and substr(dataset,1,1) in ("X","Y","Z");
  quit;
```

```
  %if &custcnt. ne 0 %then %do;

      /* Build variables by adapting RP/CM/DS model variables. */
      data dummy_custom;
          set variables_ig_mod(where=(upcase(dataset_name) in ('RP','CM','DS')) );
          do i=1 to countw("&domains", ' ');
          parent = scan("&domains", i, ' ');
          dataset_name = parent;
          if substr(variable_name,1,2) in ("RP","CM","DS") then
          variable_name = cats(parent, strip(substr(variable_name,3)));
          else variable_name = variable_name;
          output;
          end;
      run;

      /* Build SUPPQUAL templates for custom domains. */
      data suppdummy_custom;
          set variables_ig_mod(where=(upcase(dataset_name)='SUPPQUAL'));
          do i=1 to countw("&domains", ' ');
          parent = scan("&domains", i, ' ');
          dataset_name = cats('SUPP', parent);
          output;
          end;
      run;

      data expanded_ig_vars;
          set variables_ig_expanded_AP suppdummy_main dummy_custom suppdummy_custom;
      proc sort;
          by dataset_name class;
      run;

  %end;

  %if &custcnt. eq 0 %then %do;
      data expanded_ig_vars;
          set variables_ig_expanded_AP suppdummy_main;
      proc sort;
          by dataset_name class;
      run;
  %end;


  /*=====================================================================
  STEP 20: BUILD THE INITIAL VARIABLES SPECIFICATION
  ---------------------------------------------------------------------
  Merge expanded IG variables with validated domains. Populate the main
  Variables-sheet columns and preserve the standard IG variable order.

  Standard Codelist values are already present from STEP 3.
  =====================================================================*/
  proc sort data = finaldomains;
      by dataset class;
  run;

  data finvars;
      length length Dataset Variable Label Data_type Length Significant_Digits Format Mandatory
Assigned_Value Codelist Common Origin Source Pages Method Predecessor Role Has_No_Data Comment
Developer_Notes Programming_Instructions dataset_name class $32000;

      merge expanded_ig_vars(in=b) finaldomains(in=a rename=(dataset=dataset_name));
      by dataset_name class;
      if a and b;

      /* Remove study-specific standard-domain wording from custom labels. */
      if substr(Dataset_name,1,1) in ("X","Y","Z") then do;
          variable_label = compbl(tranwrd(variable_label,"Reproductive"," ") );
          variable_label = compbl(tranwrd(variable_label,"Disposition"," ") );
          codelist = "" ;
      end;

      order = input(variable_order,best.);
      Dataset = Dataset_name;
      Variable = Variable_name;
      label = variable_label;

      /* Treat DTC variables as Date variables. */
      if length(variable) gt 3 and substr(variable,length(strip(variable))-3) = "DTC" then Data_Type =
"Date";
      else Data_Type = type;
```

```
    /* Initialize specification fields that require later review/population. */
            call missing(
                length,
                Significant_Digits,
                Format,
                Mandatory,
                Assigned_Value,
                common,
                origin,
                source,
                pages,
                Method,
                Predecessor,
                Has_No_Data,
                Comment,
                Developer_Notes,
                Programming_Instructions
            );


    keep Order Dataset Variable Label Data_type Length Significant_Digits Format Mandatory Assigned_Value
Codelist Common Origin Source Pages Method Predecessor Role Has_No_Data Comment Developer_Notes
Programming_Instructions core;
    if missing(order) then order=999999;
  proc sort;
    by dataset order;
  run;


  /*=====================================================================
  STEP 21: IDENTIFY SUPP QNAM AND TESTCD ANNOTATIONS
  ---------------------------------------------------------------------
  Some annotations do not represent ordinary variables. They indicate
  value-level metadata conditions, particularly:

  - SUPP QNAM/QVAL relationships
  - Findings TESTCD/ORRES relationships

  These records are extracted separately and later converted into VLM.
  AP duplicates are also created for applicable annotations.
  =====================================================================*/

  data qnams;
    set anno;
    where (index(contents,"SUPP") and findw(upcase(contents),"IN") )
    or (index(contents,"SUPP") and findw(upcase(contents),"WHERE") );

    i=1;
    j=1;

    do while(scan(contents,i,' ') ne '');
    if upcase(scan(contents,i,' '))='IN' then do;
        dataset = scan(contents,i+1,' ');
        variable = scan(contents,1,' ');
        leave;
    end;

    if upcase(scan(contents,i,' '))='WHERE' then do;
        dataset = scan(contents,i-1,' ');
        do while(scan(contents,j,' ') ne '');
        if upcase(scan(contents,j,' '))='QNAM' then do;
            variable = scan(contents,j+1,' ');
            leave;
        end;
        j+1;
    end;
    leave;
    end;
    i+1;
    end;

    order = _n_;
    output;

    /* Duplicate annotation for corresponding SUPP AP context. */
    dataset  = "SUPPAP" || strip(substr(dataset,5));
    variable = variable;
    src  = "DUPLICATED";
    output;
  run;
```

```
/* Extract TESTCD-driven value-level annotations. */
data testcds;
    set anno;
    where (index(line_new,"TESTCD") and index(upcase(line_new),"ORRES") )
    or (index(line_new,"TESTCD") and findw(upcase(line_new),"WHERE") )
    or (index(line_new,"TESTCD") and index(upcase(line_new),"=") );

    /* Locate TESTCD within the annotation text. */
    i = 1;
    do while(scan(contents,i,' ') ne '');
    if index(upcase(scan(contents,i,' ')),'TESTCD') > 0 then leave;
    i + 1;
    end;

    /* Extract the two-letter domain and TESTCD value/condition. */
    do while(scan(contents,i,' ') ne '');
    dataset  = strip(substr(scan(contents,i,' '),1,2));
    variable = strip(scan(contents,i+1,' '));
    leave;
    i + 1;
    end;

    output;

    /* AP duplicate. */
    dataset  = "AP" || strip(dataset);
    variable = variable;
    src  = "DUPLICATED";
    output;
run;


/*=====================================================================
STEP 22: IDENTIFY NON-STANDARD / MODEL VARIABLES
---------------------------------------------------------------------
If an annotation references a model/custom variable not already found
in FINVARS, generate possible domain-variable combinations from the
standard IG model and retain matches found in the aCRF annotations.
=====================================================================*/
proc sort data = variables_ig_mod out=for_dummy_model;
    by dataset_name variable_label;
    where class not in ("Study Reference","Special-Purpose");
run;

data dummy_model_perm;
    set for_dummy_model(keep=dataset_name variable_label);

    do i=1 to nobs;
    set for_dummy_model(keep=variable_name variable_label) point=i nobs=nobs;
    newname = cats(dataset_name, substr(variable_name,3));
    output;
    end;
run;

proc sort nodupkey;
    by newname;
run;

proc sort data=anno;
    by newname;
run;

/* Build a list of standard variables already recognized. */
proc sql noprint;
    select quote(strip(variable)) into: nonmodel separated by "," from finvars ;
quit;

/* Match annotations against possible model variables. */
data model_vars;
    merge anno(in=b) dummy_model_perm(in=a keep=newname variable_label);
    by newname;

    if a and b
    and newname not in (&nonmodel.)
    and index(contents,"SUPP") eq 0
    and index(contents,"TESTCD") eq 0 ;

    dataset  = strip(substr(newname,1,2));
    variable = strip(newname);
    modelvar = "Y";
    label = variable_label;
```

```
    keep dataset variable modelvar label;
proc sort nodupkey;
    by dataset variable modelvar label;
run;


/*=====================================================================
STEP 23: COMBINE STANDARD VARIABLES WITH VLM TRIGGERS
---------------------------------------------------------------------
QNAM and TESTCD records are added to the variable-level working data
and flagged with SORTER=ZZVALUELEVEL so they can later be separated
into ValueLevel output.
=====================================================================*/
data valuelevel;
    set qnams testcds;
    keep dataset variable src;
proc sort nodupkey;
    by dataset variable;
run;

data valuelevel;
    set valuelevel;
    order= _n_;
run;

data finvars_aft_valuelevel;
    set finvars valuelevel(in=b) model_vars;
    if b then sorter = "ZZVALUELEVEL";
run;


/*=====================================================================
STEP 24: ASSIGN CRF PAGE NUMBERS AND ORIGIN
---------------------------------------------------------------------
For each recognized variable/annotation:
- Capture the CRF page.
- Set Origin=CRF.
- Deduplicate variable/page pairs.
- Transpose multiple pages into a single page list.
=====================================================================*/
proc sql ;
    select quote(strip(variable)) into: finvars separated by "," from finvars_aft_valuelevel;
quit;

data assign_page check;
    set anno;
    pageno = input(page,best.);

    if newname in (&finvars.) then do;
        output assign_page;
        origin = 'CRF';
        if origin = "CRF";
    end;

    if newname not in (&finvars.) then do;
        output check;
        origin = 'CRF';
        if origin = "CRF";
    end;

proc sort data = assign_page nodupkey;
    by newname pageno;
run;

proc sort data = check nodupkey;
    by newname pageno;
run;

/* Collapse pages into one record per variable. */
proc transpose data = assign_page out=pages prefix=pageno_;
    by newname;
    var page;
run;

/* Collapse origin values into one record per variable. */
proc sort data = assign_page out=dup_orig nodupkey;
    by newname origin;
run;

proc transpose data = dup_orig out=origin prefix=originold_;
```

```
    by newname;
    var origin;
run;

/* Merge page and origin data back to the variable metadata. */
proc sort data = finvars_aft_valuelevel;
    by variable;
run;

data page_assigned;
    length variable origin pageold $32000;
    merge finvars_aft_valuelevel pages(rename=(newname=variable)) origin(rename=(newname=variable));
    by variable;

    pageold = compbl(catx(", ",of pageno_:));
    origin = compbl(catx(", ",of originold_:));

proc sort;
    by dataset order;
run;


/*=====================================================================
STEP 25: VALIDATE VARIABLE PAGE NUMBERS AGAINST DOMAIN PAGES
---------------------------------------------------------------------
Keep only variable page numbers that are also valid domain-level pages.
This prevents page references generated from unrelated annotations from
appearing in the final specification.

Additional cleanup removes invalid duplicated/VLM annotations and
suppresses page references for SUPP IDVAR variables.
=====================================================================*/
data page_instr;
    length Programming_Instructions $32000;
    merge page_assigned(drop=pageno: originold:) finaldomains(keep=dataset page class);
    by dataset;

    length new_pageold $200 check $20;
    new_pageold = "";

    do i = 1 by 1 while(scan(pageold, i, ',') ne "");
    check = strip(scan(pageold, i, ','));
    found = 0;

    do j = 1 by 1 while(scan(page, j, ',') ne "");
    if check = strip(scan(page, j, ',')) then do;
        found = 1;
        leave;
    end;
    end;

    if found then do;
        if new_pageold = "" then
        new_pageold = check;
        else
        new_pageold = compbl(translate(catx(', ', new_pageold, check)," ",".,"));
    end;
    end;

    /* Remove duplicated records without a valid page. */
    if (strip(new_pageold)="." or strip(new_pageold)="") and src = "DUPLICATED" then delete;

    /* Remove VLM records without a valid page. */
    if (strip(new_pageold)="." or missing(new_pageold)) and sorter = "ZZVALUELEVEL" then delete;

    /* SUPP IDVAR metadata should not receive a CRF page number. */
    if substr(dataset,1,4) eq "SUPP" and index(variable,"IDVAR") then call missing(new_pageold);
run;


/*=====================================================================
STEP 26: REMOVE VARIABLES FOR DOMAINS THAT DO NOT EXIST
=====================================================================*/
proc sort data = page_instr;
    by dataset;
run;

proc sort data = finaldomains ;
    by dataset;
run;
```

```
  data assign_all1;
      merge page_instr(in=a) finaldomains(in=b keep=dataset);
      by dataset;

      if a and not b and dataset not in ("TA","TE","TI","TS") then delete;
      if missing(order) then order = 999999 ;
  run;


  /*=====================================================================
  STEP 27: SPLIT FINAL VARIABLE METADATA AND VLM TRIGGER RECORDS
  ---------------------------------------------------------------------
  Create:
  FINAL_SPEC -> Variables sheet
  VLM        -> working records used to build ValueLevel sheet
  =====================================================================*/
  data final_spec vlm;
      retain order Dataset Variable Label Data_type Length Significant_Digits Format Mandatory
Assigned_Value Codelist Common Origin Source Pages Method Predecessor Role Has_No_Data Comment
Developer_Notes Programming_Instructions ;

      set assign_all1;

      order = order;
      Dataset = Dataset;
      Variable = Variable;
      label = label;
      Data_Type = Data_Type;
      length = " ";
      Significant_Digits = " ";
      Format = "";
      Mandatory = "";
      Assigned_Value = "";
      codelist  = codelist;
      common = "";
      origin = origin;
      source = "";
      pages = new_pageold;
      Method = Method;
      Predecessor = "";
      Role = role;
      Has_No_Data = "";
      Comment = comment;
      Developer_Notes = Developer_Notes;
      Programming_Instructions = Programming_Instructions;

      if sorter eq  "ZZVALUELEVEL" then output vlm ;
      else output final_spec;

      keep Order Dataset Variable Label Data_type Length Significant_Digits Format Mandatory Assigned_Value
Codelist Common Origin Source Pages Method Predecessor Role Has_No_Data Comment Developer_Notes
Programming_Instructions core;
  run;


  /*=====================================================================
  STEP 28: PREPARE CONTROLLED TERMINOLOGY FOR TEST CODE / TEST NAME
  ---------------------------------------------------------------------
  Identify the CT codelists used for Test Code and Test Name. These
  values will later populate the VLM Value/Label information.
  =====================================================================*/
  data ctvalue;
      set ct;
      rename CDISC_Submission_Value=variable;
      if index(codelist_name,"Test Code") or index(codelist_name,"Test Name");
      if index(codelist_name,"Test Code") then sort=1;
      else if index(codelist_name,"Test Name") then sort=2;
  proc sort nodupkey;
      by variable   ;
  run;


  /*=====================================================================
  STEP 29: MATCH VLM VARIABLES TO CONTROLLED TERMINOLOGY
  ---------------------------------------------------------------------
  Match generated VLM variable names with CT variables. Retain the CT
  codelist code and individual coded values needed to enrich VLM.
  =====================================================================*/
  proc sort data = vlm out=vlmsort;
      by variable;
      where index(dataset,"SUPP") eq 0;
```

```
run;

data ct1  ;
    merge vlmsort(in=a) ctvalue(keep=code variable Codelist_Code);
    by variable;
    if a;
    keep variable Codelist_Code code;
proc sort;
    by variable Codelist_Code;
run;

proc sort data = ct1 out=coded;
    by code;
    where not missing(code);
run;

/* Build the list of CT codes that are referenced by VLM. */
proc sql noprint;
    select quote(code) into:codes separated by "," from coded
    where not missing(code);
quit;


/*=====================================================================
STEP 30: RESHAPE CT VALUES INTO VALUE/LABEL PAIRS
---------------------------------------------------------------------
For each CT code, transpose the Test Code/Test Name information into
a structure that can be merged back into the VLM records.
=====================================================================*/
proc sort data = ctvalue;
    by code sort;
run;

proc transpose data = ctvalue out=trancoded(rename=(col1=variable col2=label));
    by code;
    copy Codelist_Code;
    var variable;
    where code in (&codes.);
run;


/*=====================================================================
STEP 31: MERGE CONTROLLED TERMINOLOGY VALUES/LABELS INTO VLM
=====================================================================*/
proc sort data = trancoded ;
    by variable;
    where not missing(_name_);
run;

proc sort data = vlm ;
    by variable;
run;

data vlm_int;
    merge vlm trancoded;
    by variable;
proc sort;
    by dataset ;
run;


/*=====================================================================
STEP 32: BUILD THE FINAL VALUE-LEVEL METADATA STRUCTURE
---------------------------------------------------------------------
Convert the annotation variable into the Define-style target variable
and build the appropriate WHERE CLAUSE:

SUPPxx -> QVAL, controlled by QNAM
Standard findings -> xxORRES, controlled by xxTESTCD
AP findings -> parent domain ORRES, controlled by parent TESTCD

Also populate Origin=CRF where a page is available.
=====================================================================*/
data vlm_final_;
    length Where_Clause variable $100.;
    set vlm_int(rename=(variable=var));

    if index(dataset,"SUPP") then do;
        variable = "QVAL";
        Where_Clause =  "QNAM EQ " || strip(var);
    end;
```

```
    else do;
        if substr(dataset,1,2) = "AP" then do;
            variable = strip(substr(dataset,3)) || "ORRES";
            Where_Clause = strip(substr(dataset,3)) || "TESTCD EQ " || strip(var);
        end;

        if length(dataset) eq 2 then do;
            variable = strip(dataset) || "ORRES";
            Where_Clause = strip(dataset) || "TESTCD EQ " || strip(var);
        end;
    end;

    Data_Type = "Text";
    Length = "" ;
    Significant_Digits= "";
    Format= "";
    Mandatory= "";
    Assigned_Value= "";
    Codelist= "";

    if not missing(pages) then origin = "CRF";
    else origin = origin;

    Source = "";
    pages = pages;
    method = "";
    Predecessor = "";
    comment = "";
    Developer_Notes = "" ;

    keep Dataset Variable Where_Clause Label Data_Type Length Significant_Digits
    Format Mandatory Assigned_Value Codelist Origin Source Pages Method Predecessor Comment
    Developer_Notes;
run;


/*=========================================================================
STEP 33: ASSIGN SEQUENTIAL ORDER TO FINAL VLM
=========================================================================*/
data vlm_final;
    set vlm_final_;
    by dataset  ;
    if first.dataset then order = 0;
    order+1;
run;


/*=========================================================================
STEP 34: OPEN THE OUTPUT EXCEL WORKBOOK
=========================================================================*/
ods excel file="&outexcel..xlsx";


/*=========================================================================
STEP 35: CREATE A REUSABLE EXCEL EXPORT MACRO
-------------------------------------------------------------------------
EXPORT_SHEET handles formatting for all three output sheets.

It dynamically:
- reads metadata from PROC CONTENTS,
- identifies character variables,
- calculates observed maximum lengths,
- assigns display labels and column order,
- writes the data using PROC REPORT,
- applies review highlighting.

Review highlighting:
- Yellow = ORDER=999999, indicating a review item.
- Light red = PERM variable without CRF page information.
=========================================================================*/
%macro export_sheet(ds=,sheet=,labeltype=);

    /*-----------------------------------------------------------------
    STEP 35A: READ DATASET METADATA
    -----------------------------------------------------------------*/
    proc contents data=&ds
        out=_meta(keep=name varnum type)
        noprint;
    run;

    /* Find character variables whose actual data widths should be measured. */
```

```
    proc sql noprint;
        select name
        into :charvars separated by ' '
        from _meta
        where type=2;
    quit;

    /*---------------------------------------------------------------
    STEP 35B: CALCULATE ACTUAL MAXIMUM CHARACTER LENGTHS
    ---------------------------------------------------------------*/
    data widths;
        set &ds end=eof;
        %let nchar=%sysfunc(countw(&charvars));
        array ch {*} &charvars;
        array m {*} %do i=1 %to &nchar; m&i %end;;

            retain %do i=1 %to &nchar; m&i 0 %end;;

                do i=1 to dim(ch);
                m{i}=max(m{i},lengthn(strip(ch{i})));
            end;

            if eof then do i=1 to dim(ch);
            name=vname(ch{i});
            maxlen=m{i};
            output;
        end;

        keep name maxlen;
    run;

    proc sort data=widths;
        by name;
    run;

    proc sort data=_meta;
        by name;
    run;

    /*---------------------------------------------------------------
    STEP 35C: ASSIGN DISPLAY LABELS AND COLUMN SEQUENCE
    ---------------------------------------------------------------*/
    data _meta;
        length lbl name $200;
        merge _meta widths;
        by name;

        /*---------------------------- DATASETS ----------------------*/
        %if %upcase(&labeltype)=DATASETS %then %do;
            select (upcase(name));
                when ('DATASET')       do; seq=1; lbl='Dataset'; end;
                when ('LABEL')         do; seq=2; lbl='Label'; end;
                when ('CLASS')         do; seq=3; lbl='Class'; end;
                when ('SUBCLASS')      do; seq=4; lbl='SubClass'; end;
                when ('STRUCTURE')     do; seq=5; lbl='Structure'; end;
                when ('KEY_VARIABLES') do; seq=6; lbl='Key Variables'; end;
                when ('STANDARD')      do; seq=7; lbl='Standard'; end;
                when ('HAS_NO_DATA')   do; seq=8; lbl='Has No Data'; end;
                when ('REPEATING')     do; seq=9; lbl='Repeating'; end;
                when ('REFERENCE_DATA')do; seq=10; lbl='Reference Data'; end;
                when ('COMMENT')       do; seq=11; lbl='Comment'; end;
                when ('DEVELOPER_NOTES') do; seq=12; lbl='Developer Notes'; end;
                when ('PAGE')          do; seq=13; lbl='page'; end;
                otherwise delete;
                end;
                %end;

                /*---------------------------- VARIABLES ---------------------*/
                %else %if %upcase(&labeltype)=VARIABLES %then %do;
                    select (upcase(name));
when ('ORDER')                  do; seq=1; lbl='Order'; end;
when ('DATASET')                do; seq=2; lbl='Dataset'; end;
when ('VARIABLE')               do; seq=3; lbl='Variable'; end;
when ('LABEL')                  do; seq=4; lbl='Label'; end;
when ('DATA_TYPE')              do; seq=5; lbl='Data Type'; end;
when ('LENGTH')                 do; seq=6; lbl='Length'; end;
when ('SIGNIFICANT_DIGITS')     do; seq=7; lbl='Significant Digits'; end;
when ('FORMAT')                 do; seq=8; lbl='Format'; end;
when ('MANDATORY')              do; seq=9; lbl='Mandatory'; end;
when ('ASSIGNED_VALUE')         do; seq=10; lbl='Assigned Value'; end;
when ('CODELIST')               do; seq=11; lbl='Codelist'; end;
```

```
 when ('COMMON')                 do; seq=12; lbl='Common'; end;
 when ('ORIGIN')                 do; seq=13; lbl='Origin'; end;
 when ('SOURCE')                do; seq=14; lbl='Source'; end;
 when ('PAGES')                 do; seq=15; lbl='Pages'; end;
 when ('METHOD')                do; seq=16; lbl='Method'; end;
 when ('PREDECESSOR')            do; seq=17; lbl='Predecessor'; end;
 when ('ROLE')                   do; seq=18; lbl='Role'; end;
 when ('HAS_NO_DATA')            do; seq=19; lbl='Has No Data'; end;
 when ('COMMENT')               do; seq=20; lbl='Comment'; end;
 when ('DEVELOPER_NOTES')        do; seq=21; lbl='Developer Notes'; end;
 when ('PROGRAMMING_INSTRUCTIONS') do; seq=22; lbl='Programming Instructions'; end;
 when ('CORE')                  do; seq=23; lbl='Core'; end;
 otherwise delete;
 end;
 %end;

 /*------------------------------ VLM -------------------------*/
 %else %if %upcase(&labeltype)=VLM %then %do;
     select (upcase(name));
when ('ORDER')             do; seq=1; lbl='Order'; end;
when ('DATASET')           do; seq=2; lbl='Dataset'; end;
when ('VARIABLE')          do; seq=3; lbl='Variable'; end;
when ('WHERE_CLAUSE')      do; seq=4; lbl='Where Clause'; end;
when ('LABEL')             do; seq=5; lbl='Label'; end;
when ('DATA_TYPE')         do; seq=6; lbl='Data Type'; end;
when ('LENGTH')            do; seq=7; lbl='Length'; end;
when ('SIGNIFICANT_DIGITS')do; seq=8; lbl='Significant Digits'; end;
when ('FORMAT')            do; seq=9; lbl='Format'; end;
when ('MANDATORY')         do; seq=10; lbl='Mandatory'; end;
when ('ASSIGNED_VALUE')    do; seq=11; lbl='Assigned Value'; end;
when ('CODELIST')          do; seq=12; lbl='Codelist'; end;
when ('ORIGIN')            do; seq=13; lbl='Origin'; end;
when ('SOURCE')            do; seq=14; lbl='Source'; end;
when ('PAGES')             do; seq=15; lbl='Pages'; end;
when ('METHOD')            do; seq=16; lbl='Method'; end;
when ('PREDECESSOR')       do; seq=17; lbl='Predecessor'; end;
when ('COMMENT')           do; seq=18; lbl='Comment'; end;
when ('DEVELOPER_NOTES')   do; seq=19; lbl='Developer Notes'; end;
otherwise delete;
end;
%end;

if missing(maxlen) then maxlen=10;
width=min(max(maxlen/10,1),8);
run;

proc sort data=_meta;
    by seq;
run;

/* Freeze the header row in the Excel worksheet. */
ods excel options(sheet_name="&sheet" frozen_headers="1" flow="off");

/*-------------------------------------------------------------------
STEP 35D: GENERATE PROC REPORT DYNAMICALLY
-------------------------------------------------------------------*/
data _null_;
    set _meta end=eof;

    if _n_=1 then do;
        call execute(
        'proc report data='||strip("&ds")||
        ' nowd style(header)=[background=cxF39C12 foreground=black font_weight=bold just=l];'
        );

        %if %upcase(&labeltype)=DATASETS %then %do;
            call execute(
            'columns Dataset Label Class SubClass Structure Key_Variables Standard Has_No_Data Repeating
Reference_Data Comment Developer_Notes Page;'
            );
        %end;
        %else %if %upcase(&labeltype)=VARIABLES %then %do;
            call execute(
            'columns Order Dataset Variable Label Data_type Length Significant_Digits Format Mandatory
Assigned_Value Codelist Common Origin Source Pages Method Predecessor Role Has_No_Data Comment
Developer_Notes Programming_Instructions core;'
            );
        %end;
        %else %if %upcase(&labeltype)=VLM %then %do;
            call execute(
            'columns Order Dataset Variable Where_Clause Label Data_type Length Significant_Digits Format
```

```
Mandatory Assigned_Value Codelist Origin Source Pages Method Predecessor Comment Developer_Notes;'
            );
        %end;
    end;

    /* Create one DEFINE statement for each displayed column. */
    stmt=catx(' ','define',strip(name),'/','display',quote(trim(lbl)),
    cats('style(column)=[cellwidth=',put(width,5.1),
    'in tagattr=''Wrap:No'''];'));
    call execute(stmt);

    if eof then do;

        /* Highlight records requiring review. */
        if upcase("&labeltype") in ('VARIABLES','VLM') then do;
            call execute('compute Order;');
            call execute('if Order = 999999 then');
            call execute("call define(_row_,'style','style=[background=yellow]');");
            call execute('endcomp;');
        end;

        /* Highlight PERM variables with no CRF page reference. */
        if upcase("&labeltype") = 'VARIABLES' then do;
            call execute('compute core;');
            call execute('if order ne 9999 and upcase(strip(core)) = "PERM" and pages eq "" then');
            call execute("call define(_row_,'style','style=[background=lightred]');");
            call execute('endcomp;');
        end;

        call execute('run;');
    end;
run;

%mend export_sheet;


/*=========================================================================
STEP 36: EXPORT DATASETS SHEET
=========================================================================*/
%export_sheet(
ds=finaldomains,
sheet=Datasets,
labeltype=DATASETS
);


/*=========================================================================
STEP 37: EXPORT VARIABLES SHEET
=========================================================================*/
%export_sheet(
ds=final_spec,
sheet=Variables,
labeltype=VARIABLES
);


/*=========================================================================
STEP 38: EXPORT VALUE-LEVEL METADATA SHEET
=========================================================================*/
%export_sheet(
ds=vlm_final,
sheet=ValueLevel,
labeltype=VLM
);


/*=========================================================================
STEP 39: CLOSE THE OUTPUT EXCEL WORKBOOK
=========================================================================*/
ods excel close;

ods excel close;

/* End of main SPECS macro. */
%mend specs;


/*=========================================================================
EXAMPLE MACRO INVOCATION
-------------------------------------------------------------------------
Uncomment and update the paths/sheet names to execute the macro.
```

```
======================================================================*/
/*
%specs (
igexcel = <path>\sdtmIGfilename.xlsx,
datasetsheet = <Datasets sheet name from sdtmIG excel>,
varsheet = <Variables sheet name from sdtmIG excel >,
ctbook = <path>\Controlled Terminology workbook>,
ctsheet = <Controlled Terminology sheet name>,
outexcel = <path>\desired filename>,
fdf = <path>\fdf filename>
);
*/
```

REFERENCES

1. CDISC. Study Data Tabulation Model Implementation Guide (SDTMIG) v3.4. Published 29 November 2021.
2. CDISC. Define-XML v2.1.11. Package update published 6 April 2026.
3. CDISC. Controlled Terminology. Release dated 27 March 2026.
4. Adobe. Importing and exporting comments, Adobe Acrobat. Updated 26 February 2026.
5. Adobe. PDF Reference / Forms Data Format (FDF) documentation.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Bharath Pappula

Emmes Services Pvt Ltd.

P.Bharath7777@gmail.com

linkedin.com/in/bharath-p-b1440043